

Lab class 1 Exercise Sheet

Model answers

Question 1:

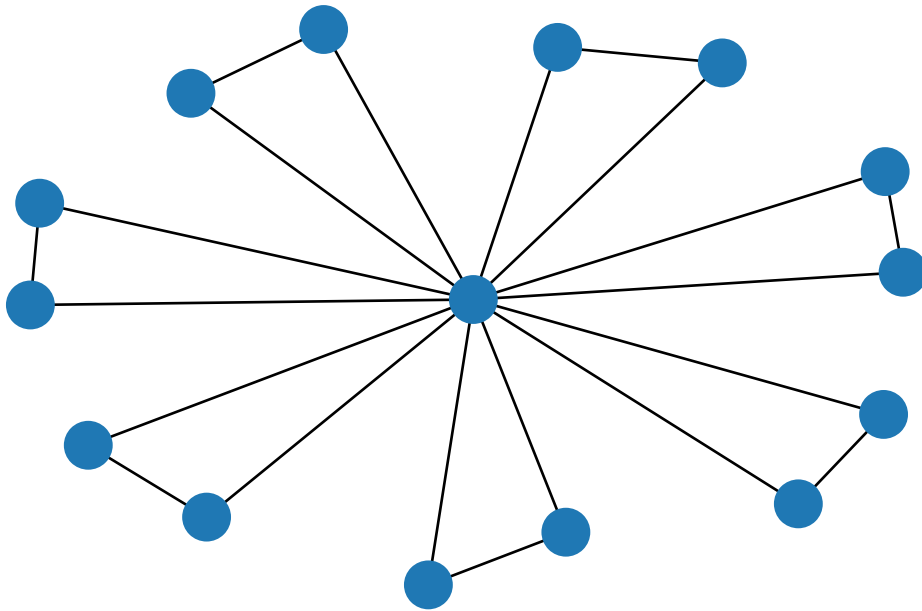
You study the mutual friendship links in your Karate club, when you notice the following: Any set of two people from the club have exactly one friend in common. What does this tell you about the club? Sketch the friendship network on paper for a club with 15 members. Once you have done that, you should be able to guess which of the [NetworkX community graph generators](#) correspond to it. Write the code to generate the friendship network for the club.

There must be someone who is friends with everybody else. This is the famous [Friendship theorem](#). You might have thought of a star network as a possible solution but it isn't because if the hub is part of the pair then there are no common friend. The only solution is therefore one where you have multiple triangles sharing a common node. This looks like the blades of a windmill and in fact the solution is the so-called [w_3 windmill graph](#).

Here is the associated code and representation. If there are 15 members, then, there are 7 blades to the windmill because each blade only contributes 2 nodes (the 3rd being the common friend).

```
import networkx as nx
import matplotlib.pyplot as plt

n_cliques = (15 - 1) // 2 # since each triangle adds only 2 new nodes
G = nx.windmill_graph(n_cliques, 3)
nx.draw(G)
plt.show()
```



💡 Karate and Network Science

[Zachary's Karate Club](#) is a very popular example of community structure in networks. There is even an award (membership of the [Zachary Karate Club](#)) for the first use of Zachary's Karate Club network in a scientific presentation.

Question 2 (tricky):

A connected simple graph has a mean degree of $z \approx 1.9990123457$. The mean degree of an undirected network is given by $k = \frac{2L}{N}$ where L is the number of links and N is the number of nodes. What is the number of nodes? (hint: draw some 'random' networks to build an intuition as to what an average degree of less than 2 could mean; you could also consult some of the network databases mentioned in the first lecture and see what typical average degrees look like.)

$N = 2025$. The key information here is the fact that the average degree is less than 2, i.e., this means that on average nodes have less than 2 neighbours. This only happens in tree networks. Now, in a tree network, the number of links is $L = N - 1$ (you have one edge between the first 2 nodes, after that you only have 1 link for each new node). So now the mean degree $z = \frac{2L}{N}$ but since $L = N - 1$, you have $z = \frac{2(N-1)}{N}$ and you get $N = 2025$.

Question 3: The Friendship paradox

The Friendship paradox [1] is the phenomenon that, on average, your friends have more friends than you (don't feel bad about it, it's a form of sampling bias). In this exercise, I am asking you to verify this empirically in random networks. So, please complete the following code skeleton (replace ... by either numbers and/or simple line of code)

```
import networkx as nx
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Generate a random graph (Erdős-Rényi model)
n = 1000 # Number of people (nodes)
p = 0.01 # Probability of forming a friendship (edge)
G = nx.erdos_renyi_graph(n, p)

# Step 2: Compute degrees of all nodes
degrees = np.array([G.degree(node) for node in G.nodes()])

# Step 3: Compute average neighbor degree for each node
neighbor_degrees = []
for node in G.nodes():
    friends = list(G.neighbors(node))
    if friends:
        avg_friend_degree = np.mean([G.degree(friend) for friend in friends])
    else:
        avg_friend_degree = 0
    neighbor_degrees.append(avg_friend_degree)

neighbor_degrees = np.array(neighbor_degrees)

# Step 4: Compute averages
overall_avg_degree = np.mean(degrees)
overall_avg_friend_degree = np.mean(neighbor_degrees)

print(f"Average degree of nodes: {overall_avg_degree:.2f}")
print(f"Average degree of friends: {overall_avg_friend_degree:.2f}")

# Step 5: Visualization
plt.figure(figsize=(8, 6))
plt.hist(degrees, bins=30, alpha=0.5, label="Node Degree")
plt.hist(neighbor_degrees, bins=30, alpha=0.5, label="Friend's Degree")
plt.axvline(overall_avg_degree,
```

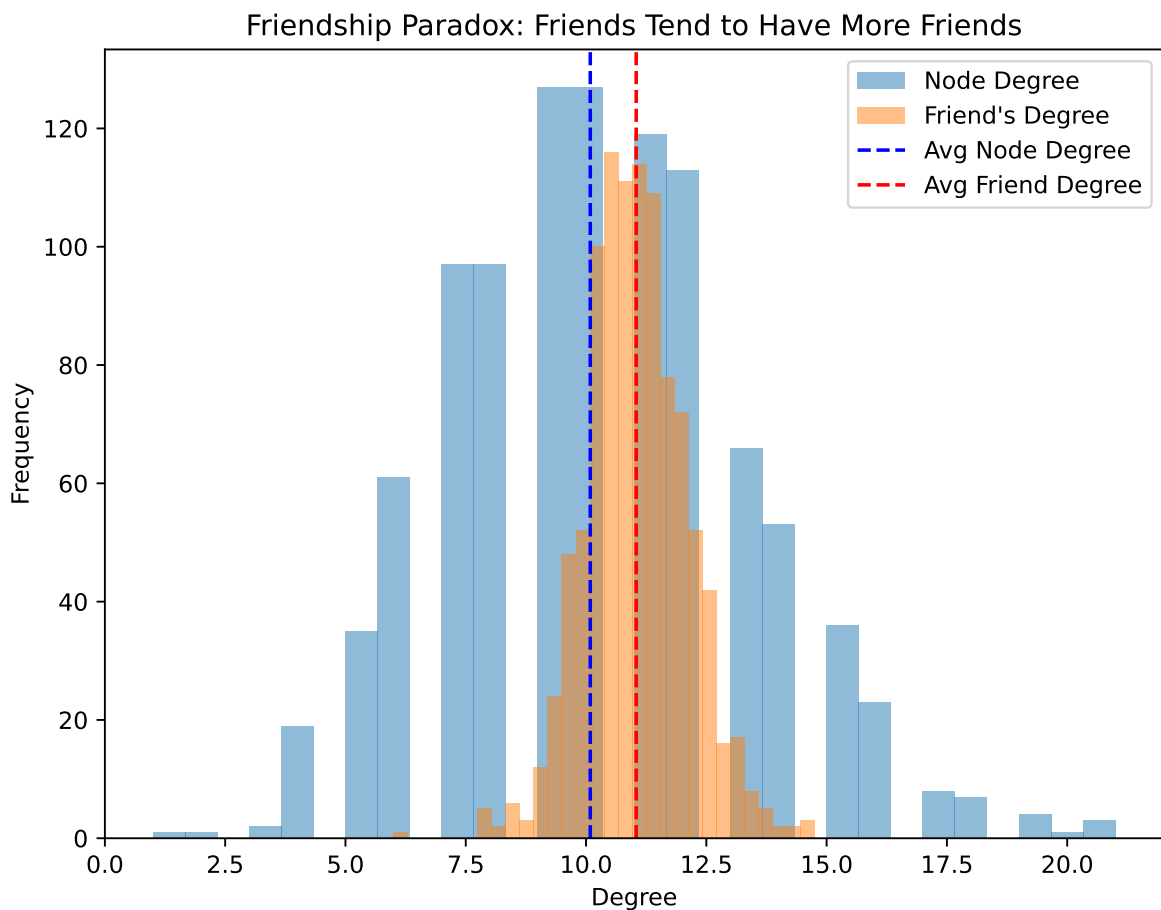
```

    color='blue', linestyle='dashed', label="Avg Node Degree")
plt.axvline(overall_avg_friend_degree,
    color='red', linestyle='dashed', label="Avg Friend Degree")
plt.xlabel("Degree")
plt.ylabel("Frequency")
plt.legend()
plt.title("Friendship Paradox: Friends Tend to Have More Friends")
plt.show()

```

Average degree of nodes: 10.09

Average degree of friends: 11.04



You will note that we used the `erdos_renyi_graph` generator. In an upcoming lecture we will discuss this graph in more detail. If you still have time in this lab class, consider using another generator, e.g., to generate a scale-free network. This would mean replacing

```
n = # Number of people (nodes)
p = # Probability of forming a friendship (edge)
G = nx.erdos_renyi_graph(n, p)
```

by

```
# Step 1: Generate a Scale-Free (Barabási-Albert) graph
n = # Number of nodes (people)
m = # Number of edges a new node attaches to existing nodes
G = nx.barabasi_albert_graph(n, m)
```

Try it and see if it makes any difference. Can you explain why?

```
import networkx as nx
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Generate a random graph (Erdős-Rényi model)
n = 1000 # Number of people (nodes)
m = 2 # Number of edges a new node attaches to existing nodes
G = nx.barabasi_albert_graph(n, m)

# Step 2: Compute degrees of all nodes
degrees = np.array([G.degree(node) for node in G.nodes()])

# Step 3: Compute average neighbor degree for each node
neighbor_degrees = []
for node in G.nodes():
    friends = list(G.neighbors(node))
    if friends:
        avg_friend_degree = np.mean([G.degree(friend) for friend in friends])
    else:
        avg_friend_degree = 0
    neighbor_degrees.append(avg_friend_degree)

neighbor_degrees = np.array(neighbor_degrees)

# Step 4: Compute averages
overall_avg_degree = np.mean(degrees)
overall_avg_friend_degree = np.mean(neighbor_degrees)
```

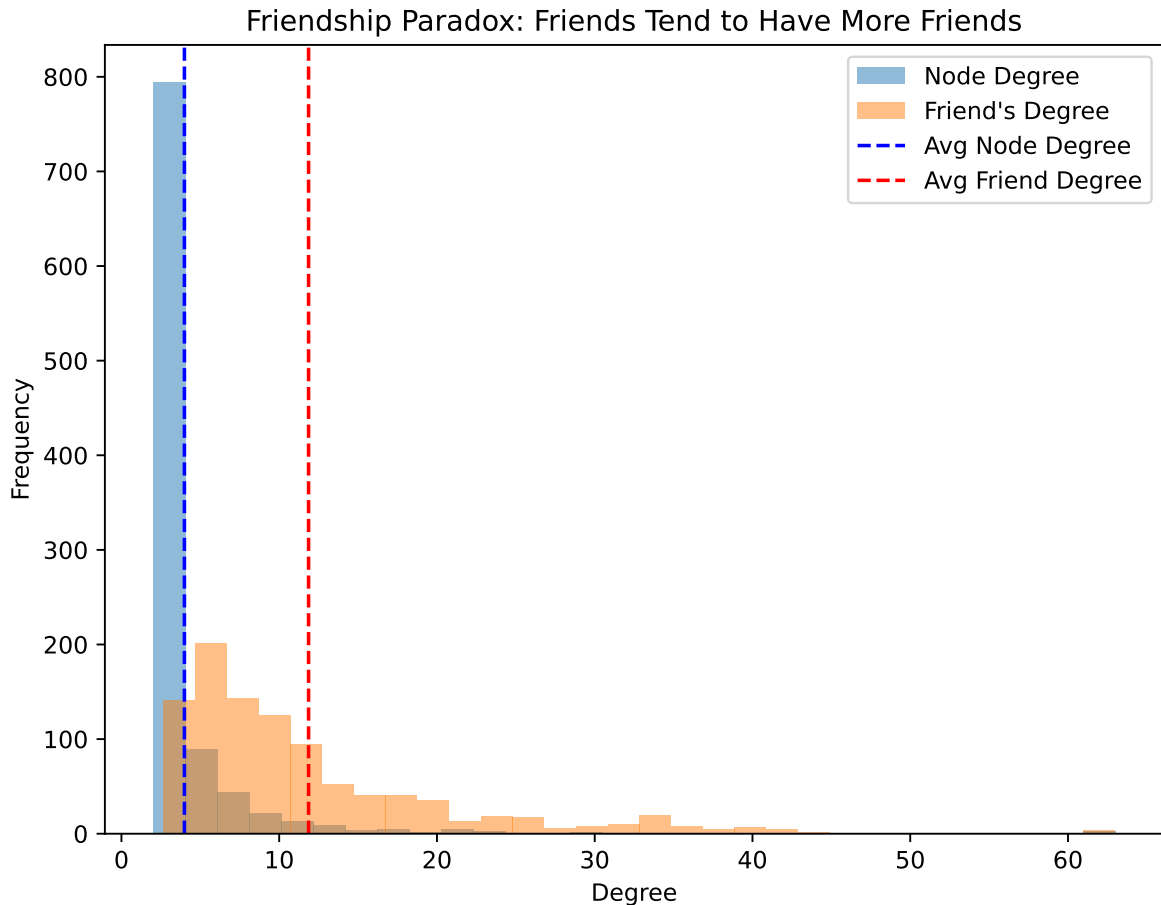
```

print(f"Average degree of nodes: {overall_avg_degree:.2f}")
print(f"Average degree of friends: {overall_avg_friend_degree:.2f}")

# Step 5: Visualization
plt.figure(figsize=(8, 6))
plt.hist(degrees, bins=30, alpha=0.5, label="Node Degree")
plt.hist(neighbor_degrees, bins=30, alpha=0.5, label="Friend's Degree")
plt.axvline(overall_avg_degree,
            color='blue', linestyle='dashed', label="Avg Node Degree")
plt.axvline(overall_avg_friend_degree,
            color='red', linestyle='dashed', label="Avg Friend Degree")
plt.xlabel("Degree")
plt.ylabel("Frequency")
plt.legend()
plt.title("Friendship Paradox: Friends Tend to Have More Friends")
plt.show()

```

Average degree of nodes: 3.99
 Average degree of friends: 11.86



Question 4: Network visualisation

In the first lecture, I briefly mentioned that one needs to be careful with network visualisation. If you still have time, consider loading a small network from one of the many network repositories (pick a network that isn't too large) and experiment with the different [layouts](#) provided by NetworkX. Remember also that within the same layout, you will get different visualisations (i.e., try the same layout many times). One useful feature of NetworkX is the ability to decide / impose positions for the nodes (i.e., you do not necessarily to let the layout function decides for you where the nodes go). A good idea then is to load file `adj.npy` containing 70 brain connectomes (each with 114 nodes). Write the code to let NetworkX decide for you how to place the nodes and then write the code to plot the nodes stored in file `coords.npy`, the 2nd scale in the Lausanne 2008 parcellation. What do you notice? (hint: you will need the function `from_numpy_array` to load the adjacency matrix into a graph; you will also want to rotate the 3D plot interactively to recognize some of the major brain structures)

Below, I only provide code for the latter part of the question, i.e., comparing the visual outputs from the spring layout and from the 3D coordinates.

```
import networkx as nx
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# Loading the adjacency matrices
adj = np.load('adj.npy')
adj = adj[:, :, 0] # arbitrarily using the first subject

# Create a graph from the adjacency matrix
G = nx.from_numpy_array(adj)

# Scenario 1: using standard networkx (spring layout default)
fig = plt.figure(figsize=(10, 8))
nx.draw(G)
plt.show()

# Scenario 2: now loading the 3D coordinates
coords = np.load('coords.npy')

# Extract node positions from the coord array
pos = {i: (coords[i, 0], coords[i, 1], coords[i, 2])
       for i in range(len(coords))}

# Initialize 3D plot
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')

# Draw edges
for i, j in G.edges():
    x = [coords[i, 0], coords[j, 0]]
    y = [coords[i, 1], coords[j, 1]]
    z = [coords[i, 2], coords[j, 2]]
    ax.plot(x, y, z, color='black', alpha=0.5, linewidth=0.7) # Edge color

# Draw nodes
x_nodes, y_nodes, z_nodes = coords[:, 0], coords[:, 1], coords[:, 2]
```



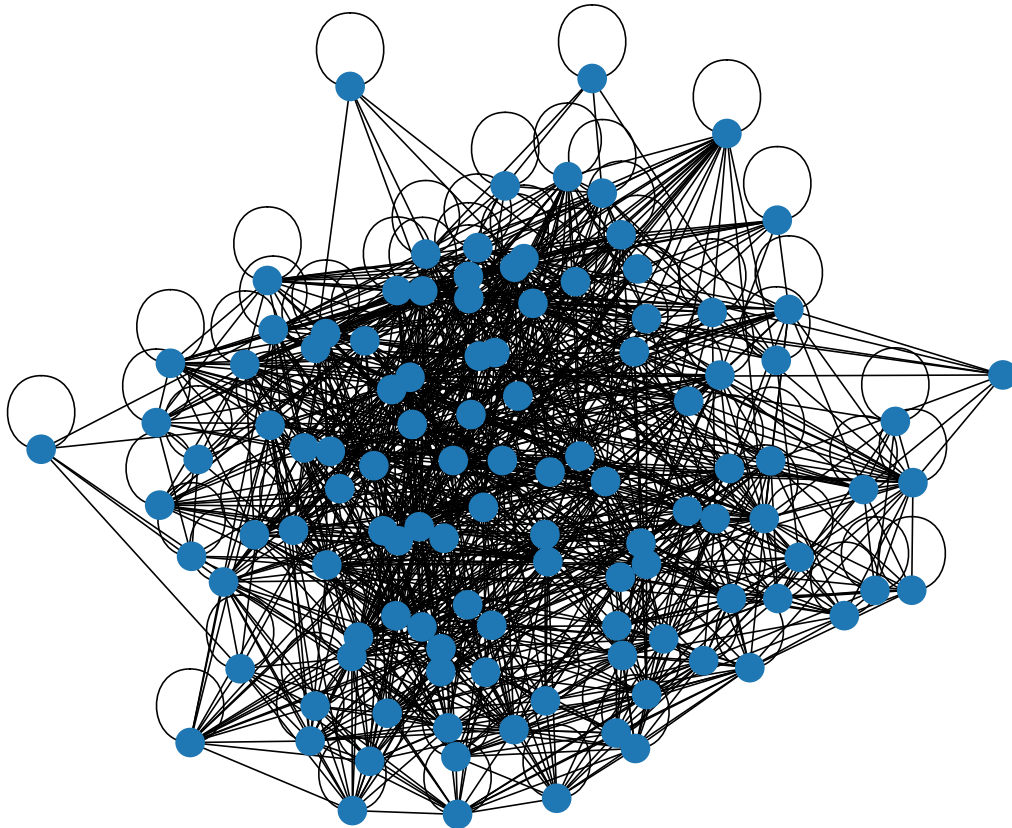
```

ax.scatter(x_nodes, y_nodes, z_nodes,
           color='red', s=20, edgecolor='black', alpha=0.8) # Node color

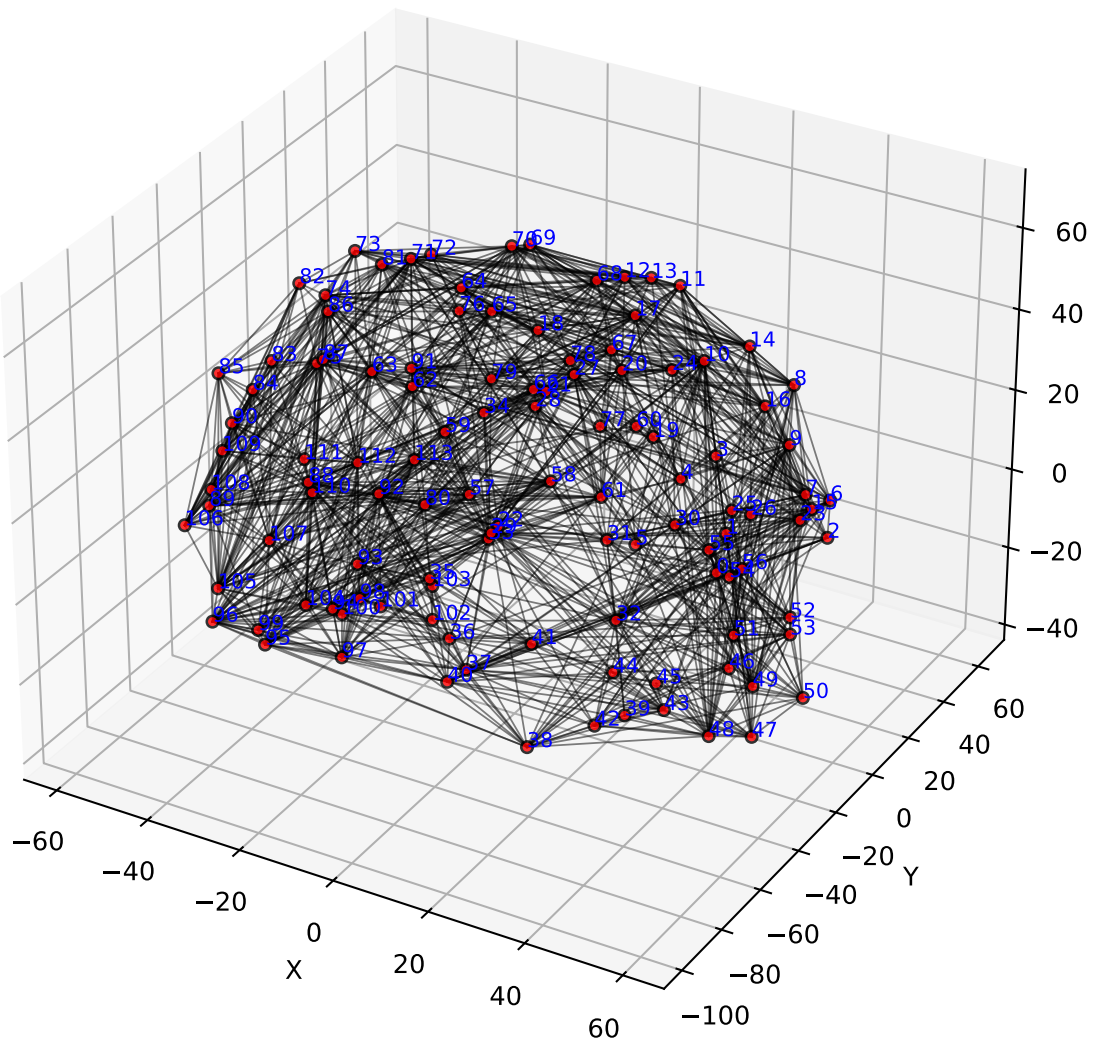
# Label nodes (optional)
for i, (x, y, z) in pos.items():
    ax.text(x, y, z, str(i), fontsize=8, color='blue')

ax.set_xlabel("X")
ax.set_ylabel("Y")
ax.set_zlabel("Z")
ax.set_title("3D Brain Connectome")
plt.show()

```



3D Brain Connectome



Obviously it is a bit of an unfair comparison. Nevertheless, the 2D representation fails to capture some of the true modularity of the brain, e.g., the fact it is made up of 2 hemispheres. This modularity becomes apparent in 3D provided you rotate the plot appropriately, as in Figure Figure 1, the 3D plot reveals the hemispheres, as well as the increased density *within* hemispheres (i.e., modularity).

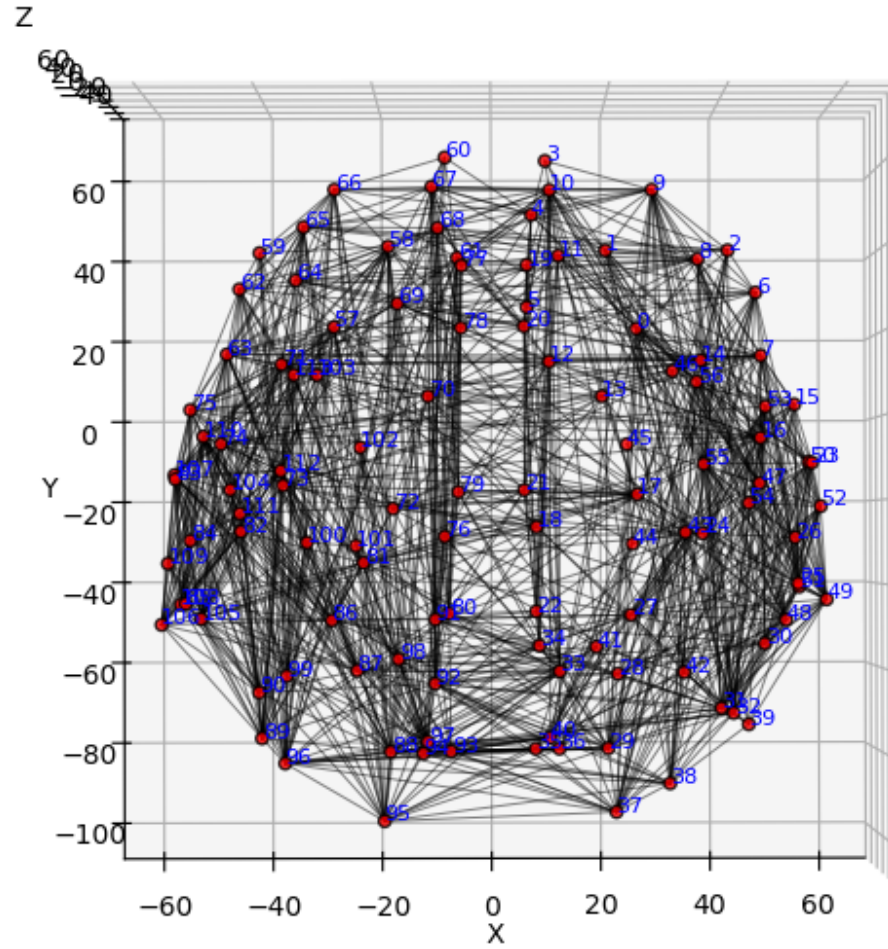


Figure 1: 3D plot projected onto the x-y plane, revealing the two hemispheres

Note

You will notice that in the 2D case, there are self-loops. An explanation is somewhat beyond the scope of this lab class but if you are curious, do feel free to ask me (i.e., Luc)

References

1. S. L. Feld, Why Your Friends Have More Friends Than You Do. *American Journal of Sociology*, **96** (1991) 1464–1477. <https://doi.org/10.1086/229693>.